Banking RAG Customer Support System

Retrieval-Augmented Generation · Human-in-the-Loop · LangGraph Orchestration

Document Type	High-Level Design (HLD)	**System**	Banking RAG Support Bot
Technology Stack	LangChain · ChromaDB · Groq · Streamlit	**LLM**	llama-3.1-8b-instant (Groq) + Falcon-RW-1B
Embedding Model	sentence-transformers/all-MiniLM-L6-v2	**Vector DB**	ChromaDB (local persistence)

Table of Contents

1 System Overview & Problem Definition
2 Architecture — Component Map
2.1 User Interface Layer
2.2 Document Ingestion Pipeline
2.3 Embedding & Vector Store
2.4 Retrieval Layer
2.5 LLM Inference Layer
2.6 HITL Routing Module
3 End-to-End Data Flow
4 Component Descriptions
5 Technology Justification
6 Scalability & Reliability Considerations
7 Security Posture

1. System Overview & Problem Definition

Modern banking institutions handle millions of customer queries daily — spanning account enquiries, KYC compliance, card management, fraud alerts, and transaction disputes. Traditional support models rely on human agents for every interaction, creating bottlenecks, escalating costs, and degrading customer experience. This document presents the High-Level Design of an **AI-powered Banking Customer Support System** built on Retrieval-Augmented Generation (RAG) with an integrated Human-in-the-Loop (HITL) safety layer.

1.1 Problem Statement

- Human agents are overwhelmed by repetitive, low-complexity queries.
- Response latency in traditional call centres degrades Net Promoter Score (NPS).
- Sensitive queries (fraud, stolen cards) require immediate routing — delays cause financial harm.
- Knowledge bases stored in static PDFs/documents are not query able by off-the-shelf chatbots.
- LLM hallucinations without grounding create compliance and liability risks in regulated sectors.

1.2 Proposed Solution

The system integrates Retrieval-Augmented Generation with Human-in-the-Loop routing to ensure both automation and safety in handling sensitive banking queries. It delivers three core capabilities:

RAG Pipeline	Grounds every LLM response in factual chunks retrieved from the official banking knowledge base, eliminating hallucinations.
HITL Routing	Rule-based intent detection intercepts fraud, card-loss, and other sensitive queries before they reach the LLM, escalating them to verified human agents.
LLM Fallback	If the primary Groq inference endpoint is unavailable, a locally hosted Falcon-RW-1B model ensures 100% uptime with graceful degradation.

1.3 Scope

In Scope	Out of Scope
✓ Natural-language query handling via Streamlit chat interface	✗ Real-time banking core API integration (account balances, transactions)
✓ PDF-based knowledge ingestion with chunk-level semantic retrieval	✗ User authentication and session management
✓ Contextual answer generation via Groq (llama-3.1-8b-instant)	✗ Transaction execution or fund transfers
✓ Rule-based HITL escalation for fraud / card-loss queries	✗ Multi-language support (v1 is English only)
✓ Persistent vector store using ChromaDB	✗ Live agent dashboard (future scope)
✓ Fallback inference using HuggingFace Falcon-RW-1B	

2. System Architecture

The system is decomposed into seven discrete layers, each with a single, well-defined responsibility. Layers communicate through standardised interfaces, enabling independent scaling and replacement.

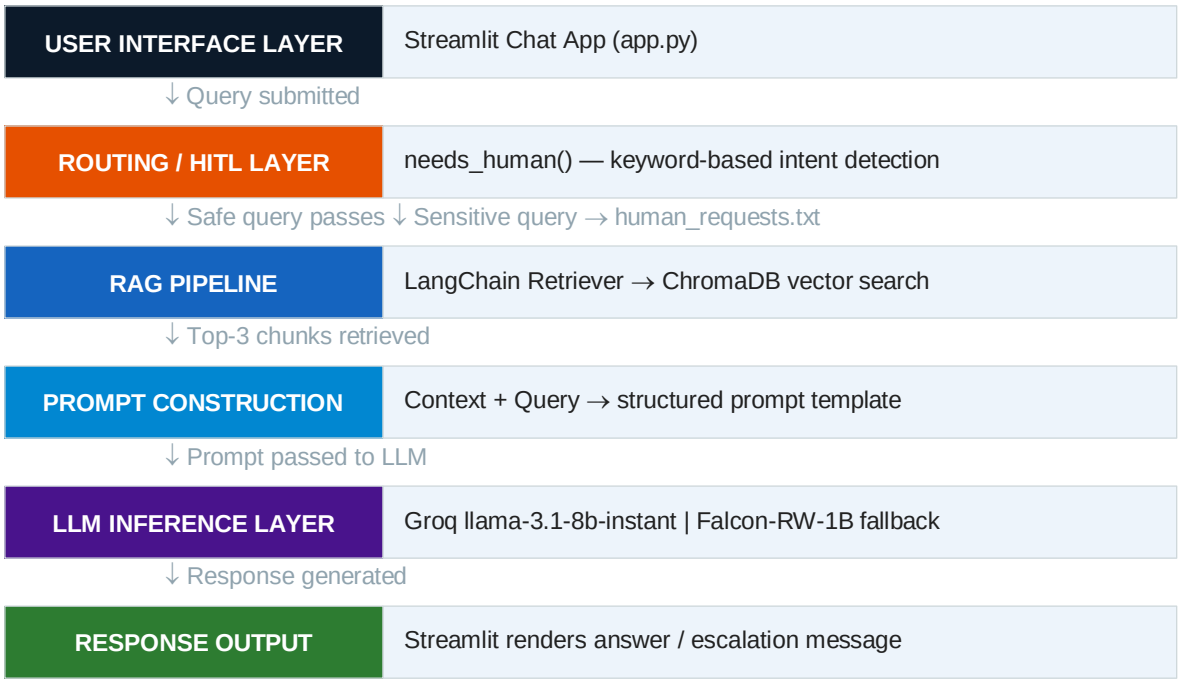

Figure 1 — System Architecture Layers (top-to-bottom query flow)

2.1 User Interface Layer — Streamlit (app.py)

- Streamlit single-page chat application renders the conversational UI.
- Maintains st.session_state.messages as the in-memory conversation history across reruns.
- Differentiates escalation responses (contains ■) from normal answers, rendering them with st.error() vs st.success().
- Clear Chat button resets session state without reloading the backend.
- Avatar URLs distinguish user and bot messages visually.

2.2 Document Ingestion Pipeline

- PyPDFLoader reads banking_customer_support_qa.pdf page-by-page.
- RecursiveCharacterTextSplitter divides raw text into 500-token chunks with 50-token overlap to preserve cross-boundary context.
- Chunks are embedded and stored in ChromaDB at ./db on first run; subsequent runs reuse the persisted store.

2.3 Embedding & Vector Store

- all-MiniLM-L6-v2 (384-dimensional dense vectors) converts each chunk into a semantic embedding.
- ChromaDB stores vectors alongside raw chunk text and page-level metadata.
- Persistence is achieved via persist_directory='./db', enabling fast cold starts.

2.4 Retrieval Layer

- vectorstore.as_retriever(search_kwargs={'k': 3}) issues a cosine-similarity search.
- Top-3 most semantically relevant chunks are concatenated as context.
- If no documents are returned (empty corpus edge case), the system short-circuits with 'I don't know'.

2.5 LLM Inference Layer

- Primary: Groq cloud API — llama-3.1-8b-instant — sub-second latency for streaming inference.
- Prompt instructs the model to answer in 2–3 lines, not repeat context, and not generate follow-up questions.
- Fallback: HuggingFace pipeline wrapping Falcon-RW-1B with max_new_tokens=150, do_sample=False for deterministic output.
- Fallback output post-processing strips the prompt prefix to return only the generated answer.

2.6 HITL Routing Module

- needs_human(query) applies two tiers of keyword matching:
- Strong triggers: 'fraud', 'unauthorized', 'stolen', 'hack' → immediate escalation regardless of other context.
- Combination triggers: 'card' AND ('lost' OR 'block') → escalation only when both signals are present, reducing false positives.
- log_human_request() appends the raw query to human_requests.txt with no PII stripping (future enhancement).
- Return value is a fixed escalation string with ■ sentinel for UI differentiation.

3. End-to-End Data Flow

3.1 Ingestion Flow (run once at startup)

Step	Action
PDF Load	PyPDFLoader reads banking_customer_support_qa.pdf → list of Document objects (one per page)
Text Split	RecursiveCharacterTextSplitter → list of chunk Documents (chunk_size=500, overlap=50)
Embed	HuggingFaceEmbeddings.embed_documents() → 384-dim float32 vectors per chunk
Store	Chroma.from_documents() writes vectors + metadata to ./db on disk
Retriever Init	vectorstore.as_retriever(k=3) returns a configured retriever object ready for queries

3.2 Query Lifecycle (per user message)

1. User Input

User types query in Streamlit chat input box.

2. HITL Check

needs_human(query) evaluates keyword rules. If True → log + return escalation message immediately.

3. Retrieval

retriever.invoke(query) calls ChromaDB cosine-similarity search; returns top-3 Document chunks.

4. Context Build

chunk.page_content values joined with double newlines → context string.

5. Prompt Assembly

Structured prompt: role-setting system instructions + context block + user question + 'Answer:' suffix.

6. LLM Call

groq_client.chat.completions.create() with model='llama-3.1-8b-instant'. On exception → fallback_llm(prompt).

7. Post-process

Response stripped of whitespace. Fallback path additionally strips the echoed prompt.

8. UI Render

Streamlit renders st.success() or st.error() based on ■ sentinel; message appended to session state.

4. Component Descriptions

Component	Technology	Role	Config
Frontend UI	Streamlit 1.x	Chat interface, session management	centered layout, avatars
Doc Loader	LangChain PyPDFLoader	Reads PDF → Document list	banking_customer_support_qa.pdf
Text Splitter	RecursiveCharacterTextSplitter	Chunks text preserving word boundaries	size=500, overlap=50
Embedding Model	all-MiniLM-L6-v2 (HuggingFace)	Dense 384-dim semantic vectors	CPU inference
Vector Store	ChromaDB	ANN similarity search + persistence	persist_dir=./db
Retriever	Chroma.as_retriever()	Top-K semantic document fetch	k=3
Primary LLM	Groq (llama-3.1-8b-instant)	Fast contextual answer generation	API key via .env
Fallback LLM	Falcon-RW-1B (HuggingFace)	Local inference on Groq failure	max_new_tokens=150
HITL Module	Custom Python (main.py)	Keyword-based escalation routing	human_requests.txt log
Orchestration	LangChain LCEL (conceptual LangGraph)	Pipeline chaining: retrieve→prompt→LLM	stateless per query
Env Config	python-dotenv	Loads GROQ_API_KEY from .env	.env file

5. Technology Justification

ChromaDB

- Zero-infrastructure setup — runs fully in-process with a local persist directory.
- Native Python SDK integrates seamlessly with LangChain's Chroma wrapper.
- Approximate nearest-neighbour (ANN) search with cosine similarity is sufficient for 500-chunk banking PDFs.
- Alternatives considered: FAISS (no persistence), Pinecone (requires cloud subscription, adds latency).

LangChain

- Provides standardised Document, VectorStore, and Retriever abstractions, reducing boilerplate.
- PyPDFLoader and RecursiveCharacterTextSplitter are battle-tested components.
- LCEL (LangChain Expression Language) enables declarative chain composition.
- Community ecosystem accelerates integration of future components (memory, agents, tools).

Groq + llama-3.1-8b-instant

- Groq's LPU inference hardware delivers sub-200ms response times — critical for conversational UX.
- llama-3.1-8b-instant achieves GPT-3.5-level quality on instruction-following benchmarks.
- Cost-effective at scale: Groq's pricing is 10–20× lower than GPT-4 Turbo for equivalent throughput.

Falcon-RW-1B Fallback

- 1-billion parameter model runs on CPU without GPU — ensures offline/fallback availability.
- do_sample=False (greedy decoding) produces deterministic, reproducible responses.
- Tradeoff: lower answer quality vs primary, but acceptable for simple FAQ-style queries.

all-MiniLM-L6-v2

- 384-dimensional embeddings strike the best balance between retrieval quality and speed.
- Trained specifically for semantic similarity tasks — outperforms general-purpose embeddings on Q&A; retrieval.
- No API calls required — inference runs locally, keeping PII within the deployment boundary.

Streamlit

- Rapid prototyping framework requiring zero frontend expertise.
- st.session_state provides conversational memory within a browser session.
- st.chat_message and st.chat_input provide native chat UI primitives.
- One-command deployment: streamlit run app.py.

6. Scalability & Reliability Considerations

Document Scale

- Current: single PDF (~hundreds of pages). ChromaDB handles millions of vectors on commodity hardware.
- Scale path: Replace local ChromaDB with Chroma Cloud or migrate to Weaviate/Qdrant for distributed search.
- Chunking strategy remains effective up to ~10,000 chunks before retrieval precision degrades — resolved by metadata filtering.

Query Throughput

- Groq API supports concurrent request batching — horizontal scale via load balancing multiple API keys.
- Streamlit sessions are independent — no shared state between users; scales linearly with container replicas.
- Future: FastAPI backend replaces Streamlit backend, enabling REST/WebSocket clients and CDN caching of common answers.

Latency Budget

- Embedding lookup: ~20–50 ms (local CPU, MiniLM).
- ChromaDB ANN search: < 10 ms for <50,000 vectors.
- Groq inference: ~150–400 ms (dependent on response length).
- Total p95 latency target: < 700 ms end-to-end for typical queries.

Reliability & Fallback

- Primary LLM failure (network, quota) caught by try/except → Falcon-RW-1B activated automatically.
- ChromaDB corruption: persist_directory can be rebuilt by re-running the ingestion pipeline.
- HITL log (human_requests.txt) is append-only — no data loss on crash.

7. Security Posture

While v1 is a prototype, the design incorporates foundational security hygiene. The following table maps current implementation state against future hardening requirements.

Concern	Current State	Recommended Enhancement
API Key Management	GROQ_API_KEY loaded from .env via python-dotenv	Rotate via secret manager (AWS Secrets Manager / Vault)
PII in Logs	Raw query written to human_requests.txt	Anonymise / redact PII before logging; encrypt log file at rest
Authentication	None — open access	OAuth2 / SSO integration; role-based access for agents
Input Validation	No sanitisation beyond keyword matching	Prompt injection detection; input length limits
Data Encryption	Plaintext ChromaDB on disk	Encrypt ./db at rest using LUKS / cloud KMS
Audit Trail	human_requests.txt append log	Structured audit log (JSON) with timestamps, user IDs, query hashes